Unit-3 (2 marks)

1.Write the any two differences between StringBuffer and StringBuilder class.

J2SE 5 adds a new string class is called **StringBuilder**. It is identical to **StringBuffer** except for one important difference: it is not synchronized, which means that it is not thread-safe. The advantage of **StringBuilder** is faster performance. However, in cases in which you are using multithreading, you must use **StringBuffer** rather than **StringBuilder**

2.Write the purpose of + operator in string handling with an example.

+ operator, which concatenates two strings, producing a String object as the result. This allows you to chain together a series of + operations. For example, the following fragment concatenates three strings:

```
String age = "9";
String s = "He is " + age + " years old.";
System.out.println(s);
```

This displays the string "He is 9 years old.

3. Write two ways to create a String object using the String constructors, and provide an example for each

- The String class supports several constructors.
- To create an empty String, you call the default constructor.
- For example, **String s = new String();**
- will create an instance of String with no characters in it.
- The String class provides a variety of constructors to create strings that have initial values.
- To create a String initialized by an array of characters, use the constructor shown here: **String(char chars[ ])**
- Here is an example: **char chars[] = { 'a', 'b', 'c' };**
- **String s = new String(chars);**
- This constructor initializes s with the string "abc"

4. What is the purpose of the toString() method in the context of strings.

The toString( ) method has this general form: **String toString()**.

To implement toString( ), simply return a String object that contains the human-readable string that appropriately describes an object of your class.

5. How to initialize String objects using string literals?

- you can use a string literal to initialize a String object.
- For example, the following code fragment creates two equivalent strings:
  char chars[] = { 'a', 'b', 'c' };
  String s1 = new String(chars);
  String s2 = "abc"; // use string literal
  Because a String object is created for every string literal, you can use a string literal any place you can use a String object.

6. What are the different ways to extract individual characters from a string? Give the syntax

charAt( )- To extract a single character from a String, you can refer directly to an individual character via the charAt( ) method. It has this general form: **char charAt(int where)** Here, where is the index of the character that you want to obtain

getChars( ) If you need to extract more than one character at a time, you can use the getChars( ) method. It has this general form: **void getChars(int sourceStart, int sourceEnd, char target[ ], int targetStart)** Here, sourceStart specifies the index of the beginning of the substring, and sourceEnd specifies an index that is one past the end of the desired substring

getBytes( ) There is an alternative to getChars( ) that stores the characters in an array of bytes. This method is called **getBytes( ),** and it uses the default character-to-byte conversions provided by the platform. Here is its simplest form: **byte[ ] getBytes( )**

toCharArray( ) If you want to convert all the characters in a String object into a character array, the easiest way is to call toCharArray( ). It returns an array of characters for the entire string. It has this general form: char[ ] toCharArray()

7. Provide the syntax and an example for the charAt() method to extract characters from a string.

charAt( )- To extract a single character from a String, you can refer directly to an individual character via the charAt( ) method. It has this general form: **char charAt(int where)** Here, where is the index of the character that you want to obtain For example, char ch; ch = "abc".charAt(1); assigns the value "b" to ch

8.Write the purpose and syntax of the regionMatches() method in string handling

- The **regionMatches( )** method compares a specific region inside a string with another specific region in another string.
- There is an overloaded form that allows you to ignore case in such comparisons.
- Here are the general forms for these two methods:
  boolean regionMatches(int startIndex, String str2, int str2StartIndex, int numChars)
- **boolean regionMatches(boolean ignoreCase, int startIndex, String str2, int str2StartIndex, int numChars)**

9. Demonstrate the usage of the startsWith() and endsWith() methods in string handling.

- The **startsWith( )** method determines whether a given String begins with a specified string.
- Conversely, **endsWith( )** determines whether the String in question ends with a specified string.
- They have the following general forms:
  boolean startsWith(String str)
  boolean endsWith(String str)
  Here, str is the String being tested.
- If the string matches, true is returned. Otherwise, false is returned. For example, **"Foobar".endsWith("bar")** and **"Foobar".startsWith("Foo")** are both true.

10.What's the difference between equals() and == ?

the equals( ) method compares the characters inside a String object. The == operator compares two object references to see whether they refer to the same instance.

```
// equals() vs == class EqualsNotEqualTo {
public static void main(String args[]) {
```

```
String s1 = "Hello"; String s2 = new String(s1);
System.out.println(s1 + " equals " + s2 + " -> " +
s1.equals(s2));
System.out.println(s1 + " == " + s2 + " -> " + (s1 == s2)); }
}output: Hello equals Hello -> true  Hello == Hello -> false
```

11. Describe the functionalities of the indexOf and lastIndexOf methods used for string manipulation

- **indexOf( )** Searches for the first occurrence of a character or substring.
- lastIndexOf( ) Searches for the last occurrence of a character or substring
- To search for the first or last occurrence of a substring, use **int indexOf(String str)**
- **int lastIndexOf(String str)** Here, str specifies the substring

```
String s = "Now is the time for all good men " + "to come to
the aid of their country.";
System.out.println("indexOf(the) = " + s.indexOf("the"));
System.out.println("lastIndexOf(the)=" + s.lastIndexOf("the"));
Output:
indexOf(the) = 7 lastIndexOf(the) = 55
```

12. What are the two forms of the substring() method and what do their arguments represent?

- **substring( )** You can extract a substring using substring( ). It has two forms.
- The first is **String substring(int startIndex)** Here, startIndex specifies the index at which the substring will begin. This form returns a copy of the substring that begins at startIndex and runs to the end of the invoking string.
- The second form of substring( ) allows you to specify both the beginning and ending index of the substring: **String substring(int startIndex, int endIndex)** Here, startIndex specifies the beginning index, and endIndex specifies the stopping point. The string returned contains all the characters from the beginning index, up to, but not including, the ending index

13. Describe the two forms of the replace() method and their functionalities.

- The **replace( )** method has two forms.
- The first replaces all occurrences of one character in the invoking string with another character.
- It has the following general form: **String replace(char original, char replacement)** Here, original specifies the character to be replaced by the character specified by replacement. The resulting string is returned. For example, String s = "Hello".replace('l', 'w'); puts the string "Hewwo" into s.
- The second form of replace( ) replaces one character sequence with another.
- It has this general form: **String replace(CharSequence original, CharSequence replacement)**

14. What is the purpose of the valueOf() method in Java and how does it relate to the toString() method?

- The **valueOf( )** method converts data from its internal format into a human-readable form.
- **valueOf( )** is also overloaded for type Object, so an object of any class type you create can also be used as an argument.
- Here are a few of its forms:
  static String valueOf(double num)
  static String valueOf(long num)
  static String valueOf(Object ob)
  static String valueOf(char chars[ ]
- Any object that you pass to valueOf( ) will return the result of a call to the object's **toString( )** method

15. How does StringBuffer differ from String in terms of mutability and growth?

- **StringBuffer** is a peer class of String that provides much of the functionality of strings.
- As you know, String represents fixed-length, immutable character sequences.
- In contrast, StringBuffer represents growable and writeable character sequences.
- StringBuffer may have characters and substrings inserted in the middle or appended to the end.

- StringBuffer will automatically grow to make room for such additions and often has more characters preallocated than are actually needed, to allow room for growth.

16. What is the purpose of the ensureCapacity() method in StringBuffer? give its general form

ensureCapacity( )-If you want to preallocate room for a certain number of characters after a StringBuffer has been constructed, you can use ensureCapacity( ) to set the size of the buffer.

- This is useful if you know in advance that you will be appending a large number of small strings to a StringBuffer.
- ensureCapacity( ) has this general form:
  void ensureCapacity(int capacity) Here, capacity specifies the size of the buffer

17. How does the setLength() method modify the length of a StringBuffer and what happens to existing data when the length is changed?

- To set the length of the buffer within a StringBuffer object, use **setLength( ).**
- Its general form is shown here: **void setLength(int len)** Here, len specifies the length of the buffer.
- This value must be nonnegative.
- When you increase the size of the buffer, null characters are added to the end of the existing buffer.
- If you call **setLength( )** with a value less than the current value returned by length( ), then the characters stored beyond the new length will be lost

18. What do the charAt() and setCharAt() methods do in StringBuffer?

- The value of a single character can be obtained from a StringBuffer via the **charAt( )** method.
- You can set the value of a character within a StringBuffer using **setCharAt( ).**
- Their general forms are shown here: **char charAt(int where) void setCharAt(int where, char ch)**

- For charAt( ), where specifies the index of the character being obtained.
- For setCharAt( ), where specifies the index of the character being set, and ch specifies the new value of that character. For both methods, where must be nonnegative and must not specify a location beyond the end of the buffer

19. What does the append() method do in StringBuffer?Which funciton is called for each parameter to obtain its string representation.

- The **append( )** method concatenates the string representation of any other type of data to the end of the invoking StringBuffer object.
- Here are a few of its forms: **StringBuffer append(String str)**
  StringBuffer append(int num)
  StringBuffer append(Object obj)
- **String.valueOf( )** is called for each parameter to obtain its string representation.
- The result is appended to the current StringBuffer object.
- The buffer itself is returned by each version of append( ).

20. What does the insert() method do in StringBuffer and how is it different from append()?

- The **insert( )** method inserts one string into another.
- It is overloaded to accept values of all the simple types, plus Strings, Objects, and CharSequences.
- Like append( ), it calls **String.valueOf( )** to obtain the string representation of the value it is called with.
- This string is then inserted into the invoking StringBuffer object.
- These are a few of its forms:
  StringBuffer insert(int index, String str)
  StringBuffer insert(int index, char ch)
  StringBuffer insert(int index, Object obj) Here, index specifies the index at which point the string will be inserted into the invoking StringBuffer object.

21. How does StringBuilder differ from StringBuffer?

J2SE 5 adds a new string class is called **StringBuilder**. It is identical to **StringBuffer** except for one important difference: it is not synchronized, which means that it is not thread-safe. The advantage of **StringBuilder** is faster performance. However, in cases in which you are using multithreading, you must use **StringBuffer** rather than **StringBuilder**

22. What are the roles of the Stub and Skeleton objects in RMI?

The communication between client and server is handled by using two intermediate objects: Stub object (on client side) and Skeleton object (on server-side)

23. What is RMI?

The RMI (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. The RMI allows an object to invoke methods on an object running in another JVM. The RMI provides remote communication between the applications using two objects stub and skeleton

24. What is Syntactic transparency?

Syntactic transparency: This implies that there should be a similarity between the remote process and a local procedure

25. What is Semantic transparency?

Semantic transparency: This implies that there should be similarity in the semantics i.e. meaning of a remote process and a local procedure.

26. What are the requirements for a class to be serializable?

- In Java, serialization is achieved by implementing the Serializable interface, which is a marker interface indicating that the class is serializable.
- Serializable objects can be written to an output stream (e.g., a file or network socket) using an **ObjectOutputStream**, and later read from an input stream using an ObjectInputStream.
- Serialization allows objects to be easily stored to disk, transmitted over a network, or saved to a database, enabling object persistence

27. **State two advantages of using distributed computing over centralized computing.**

Scalability: Distributed systems are generally more scalable than centralized systems, as they can easily add new devices or systems to the network to increase processing and storage capacity

Reliability: Distributed systems are often more reliable than centralized systems, as they can continue to operate even if one device or system fails.

Flexibility: Distributed systems are generally more flexible than centralized systems, as they can be configured and reconfigured more easily to meet changing computing needs.

28. **State any four key characteristics that define a distributed computing system.**

Multiple Devices or Systems: Processing and data storage is distributed across multiple devices or systems.

- **Peer-to-Peer Architecture:** Devices or systems in a distributed system can act as both clients and servers, as they can both request and provide services to other devices or systems in the network.
- **Shared Resources:** Resources such as computing power, storage, and networking are shared among the devices or systems in the network. •

Horizontal Scaling: Scaling a distributed computing system typically involves adding more devices or systems to the network to increase processing and storage capacity. This can be done through hardware upgrades or by adding additional devices or systems to the network.

29. **List the elements used in the working of RPC**

There are 5 elements used in the working of RPC:

- Client
- Client Stub
- RPC Runtime
- Server Stub
- Server

30. **Why is the RMI Registry important in Remote Method Invocation (RMI) applications?**

- The RMI registry is a simple naming service provided by Java RMI that allows clients to look up remote objects by name.
- It acts as a central registry where remote objects are bound to names, making them accessible to clients.
- The server creates an RMI registry, and clients use it to locate remote objects.

31. Write any two limitations of distributed computing.

Complexity: Distributed systems can be more complex than centralized systems, as they involve multiple devices or systems that need to be coordinated and managed.

- **Security:** It can be more challenging to secure a distributed system, as security measures must be implemented on each device or system to ensure the security of the entire system.
- **Performance:** Distributed systems may not offer the same level of performance as centralized systems, as processing and data storage is distributed across multiple devices or systems

32. When does RMI callback occur?

An RMI callback occurs when the client of one service passes an object, that is the proxy, for another service. The recipient can then call methods in the object it received, and be calling back (hence the name) to where it came from

(4 to 6 marks)

1. **With syntax and example explain the following String class methods a. split() b.regionMatches()**

a) **String[ ] split(String regExp)**- Decomposes the invoking string into parts and returns an array that contains the result. Each part is delimited by the regular expression passed in regExp.

b) **String[ ] split(String regExp, int max)** Decomposes the invoking string into parts and returns an array that contains the result. Each part is delimited by the regular expression passed in regExp. The number of pieces is specified by max. If max is negative, then the invoking string is fully decomposed. Otherwise, if max contains a nonzero value, the last entry in the returned array contains the remainder of the invoking string. If max is zero, the invoking string is fully decomposed.

c) **The regionMatches( )** method compares a specific region inside a string with another specific region in another string. There is an overloaded form that allows you to ignore case in such comparisons. Here are the general forms for these two methods: **boolean regionMatches(int startIndex, String str2, int str2StartIndex, int numChars)**

boolean regionMatches(boolean ignoreCase, int startIndex, String str2, int str2StartIndex, int numChars) For both versions, startIndex specifies the index at which the region begins within the invoking String object. The String being compared is specified by str2. The index at which the comparison will start within str2 is specified by str2StartIndex. The length of the substring being compared is passed in numChars. In the second version, if ignoreCase is true, the case of the characters is ignored. Otherwise, case is significant.

2. **With syntax and example explain the following String class methods a) getChars() b) replace()**

a) **getChars( )**- If you need to extract more than one character at a time, you can use the **getChars( )** method.

- It has this general form: **void getChars(int sourceStart, int sourceEnd, char target[ ], int targetStart)** Here, sourceStart specifies the index of the beginning of the substring, and sourceEnd specifies an index that is one past the end of the desired substring.

- Thus, the substring contains the characters from sourceStart through sourceEnd.
- The array that will receive the characters – is specified by target.
- The index within target at which the substring will be copied is passed in targetStart.
- The following program demonstrates getChars( ):

```
class getCharsDemo {
public static void main(String args[]) {
String s = "This is a demo of the getChars method.";
int start = 10; int end = 14;
char buf[] = new char[end - start];
s.getChars(start, end, buf, 0);
System.out.println(buf);
}
} Here is the output of this program: demo
```

b) replace( )-The replace( ) method has two forms.

- The first replaces all occurrences of one character in the invoking string with another character.
- It has the following general form:
  String replace(char original, char replacement) Here, original specifies the character to be replaced by the character specified by replacement. The resulting string is returned.
- For example, String s = "Hello".replace('l', 'w'); puts the string "Hewwo" into s.
- The second form of replace( ) replaces one character sequence with another.
- It has this general form:
  String replace(CharSequence original, CharSequence replacement)

3. With syntax and example explain the following StringBuffer class methods a. insert() b) deleteCharAt()

a) insert( ): The insert( ) method inserts one string into another.

- It is overloaded to accept values of all the simple types, plus Strings, Objects, and CharSequences.
- Like append( ), it calls String.valueOf( ) to obtain the string representation of the value it is called with.

- This string is then inserted into the invoking StringBuffer object.
- These are a few of its forms:
  StringBuffer insert(int index, String str)
  StringBuffer insert(int index, char ch)
  StringBuffer insert(int index, Object obj) Here, index specifies the index at which point the string will be inserted into the invoking StringBuffer object.
- The following sample program inserts "like" between "I" and "Java": // Demonstrate insert().

```
class insertDemo {
public static void main(String args[]) {
StringBuffer sb = new StringBuffer("I Java!");
sb.insert(2, "like ");
System.out.println(sb);
}
} The output of this example is shown here: I like Java!
```

b) deleteChartAt(): You can delete characters within a StringBuffer by using the methods delete( ) and deleteCharAt( ).

- These methods are shown here:
  StringBuffer delete(int startIndex, int endIndex)
  StringBuffer deleteCharAt(int loc)
- The delete( ) method deletes a sequence of characters from the invoking object.
- Here, startIndex specifies the index of the first character to remove, and endIndex specifies an index one past the last character to remove.
- Thus, the substring deleted runs from startIndex to endIndex–1.
- The resulting StringBuffer object is returned.
- **The deleteCharAt( )** method deletes the character at the index specified by loc.
- It returns the resulting StringBuffer object.
- Here is a program that demonstrates the delete( ) and deleteCharAt( ) methods:

```
// Demonstrate delete() and deleteCharAt()
class deleteDemo {
public static void main(String args[]) {
```

```
StringBuffer sb = new StringBuffer("This is a test.");
sb.delete(4, 7);
System.out.println("After delete: " + sb);
 sb.deleteCharAt(0);
 System.out.println("After deleteCharAt: " + sb);
}
```

} The following output is produced: After delete: This a test. After deleteCharAt: his a test

4. With syntax and example explain the following StringBuffer class methods a. substring() b)lastIndexOf()

a) substring( ) You can obtain a portion of a StringBuffer by calling substring( ).

- It has the following two forms:
  String substring(int startIndex)
  String substring(int startIndex, int endIndex)
- The first form returns the substring that starts at startIndex and runs to the end of the invoking StringBuffer object.
- The second form returns the substring that starts at startIndex and runs through endIndex–1

```
// Demonstrate Substring
 class replaceDemo {
public static void main(String args[]) {
StringBuffer sb = new StringBuffer("This is a test.");
sb.substring(5, 7);
System.out.println("After Substring: " + sb);
```

} } Here is the output: After substring: is

b)lastIndexOf()

int lastIndexOf(String str) Searches the invoking StringBuffer for the last occurrence of str. Returns the index of the match, or –1 if no match is found.

int lastIndexOf(String str, int startIndex) Searches the invoking StringBuffer for the last occurrence of str, beginning at startIndex. Returns the index of the match, or –1 if no match is found. //Demo program

```
class IndexOfDemo {
public static void main(String args[]) {
StringBuffer sb = new StringBuffer("one two one");
int i; i = sb.indexOf("one");
System.out.println("First index: " + i);
i = sb.lastIndexOf("one");
```

System.out.println("Last index: " + i);
} } The output is shown here: First index: 0 Last index: 8

5. Explain how to concatenate string with other data types?

- You can concatenate strings with other types of data.
- For example, consider this example:
  int age = 9;
  String s = "He is " + age + " years old.";
  System.out.println(s);
- In this case, age is an int rather than another String, but the output produced is the same as before.
- This is because the int value in age is automatically converted into its string representation within a String object.
- This string is then concatenated as before.
- The compiler will convert an operand to its string equivalent whenever the other operand of the + is an instance of String.
- Consider the following:
  String s = "four: " + 2 + 2;
  System.out.println(s);
  This fragment displays four: 22
  rather than the four: 4
- Operator precedence causes the concatenation of "four" with the string equivalent of 2 to take place first.
- This result is then concatenated with the string equivalent of 2 a second time.
- String s = "four: " + (2 + 2);
  Now s contains the string "four: 4"

6. Explain character extraction methods of String class with an example

charAt( ) To extract a single character from a String, you can refer directly to an individual character via the **charAt( ) method.**

- It has this general form: **char charAt(int where)** Here, where is the index of the character that you want to obtain.
- The value of where must be nonnegative and specify a location within the string.

- charAt( ) returns the character at the specified location.
- For example, char ch;
  ch = "abc".charAt(1);
  assigns the value "b" to ch.

getChars( ) If you need to extract more than one character at a time, you can use the **getChars( ) method.**

- It has this general form: **void getChars(int sourceStart, int sourceEnd, char target[ ], int targetStart)** Here, sourceStart specifies the index of the beginning of the substring, and sourceEnd specifies an index that is one past the end of the desired substring.
- Thus, the substring contains the characters from sourceStart through sourceEnd1.
- The array that will receive the characters – is specified by target. The index within target at which the substring will be copied is passed in targetStart
- The following program demonstrates getChars( ):

```
class getCharsDemo {
 public static void main(String args[]) {
String s = "This is a demo of the getChars method.";
int start = 10; int end = 14;
char buf[] = new char[end - start];
s.getChars(start, end, buf, 0);
System.out.println(buf); }
} Here is the output of this program: demo
```

7. Explain any four string comparison methods with an example

equals( ) and equalsIgnoreCase( )

- To compare two strings for equality, use equals( ). It has this general form: **boolean equals(Object str)** Here, str is the String object being compared with the invoking String object.
- It returns true if the strings contain the same characters in the same order, and false otherwise.
- The comparison is case-sensitive.
- To perform a comparison that ignores case differences, call **equalsIgnoreCase( ).**
- When it compares two strings, it considers A-Z to be the same as a-z.

- It has this general form: **boolean equalsIgnoreCase(String str)** Here, str is the String object being compared with the invoking String object.
- It, too, returns true if the strings contain the same characters in the same order, and false otherwise.
- Here is an example that demonstrates equals( ) and equalsIgnoreCase( ):

```
// Demonstrate equals() and equalsIgnoreCase().
class equalsDemo {
public static void main(String args[]) {
String s1 = "Hello"; String s2 = "Hello";
String s3 = "Good-bye";
String s4 = "HELLO";
System.out.println(s1+ " equals " + s2 + " -> " + s1.equals(s2));
System.out.println(s1+ " equals " + s3 + " -> " + s1.equals(s3));
System.out.println(s1+ " equals " + s4 + " -> " + s1.equals(s4));
System.out.println(s1+ " equalsIgnoreCase " + s4 + " -> " +
s1.equalsIgnoreCase(s4));
}
}
```

The output from the program is shown here:

```
Hello equals Hello -> true
Hello equals Good-bye -> false
Hello equals HELLO -> false
Hello equalsIgnoreCase HELLO -> true
```

regionMatches( ) The regionMatches( ) method compares a specific region inside a string with another specific region in another string.

- There is an overloaded form that allows you to ignore case in such comparisons.
- Here are the general forms for these two methods:
  boolean regionMatches(int startIndex, String str2, int str2StartIndex, int numChars)
  boolean regionMatches(boolean ignoreCase, int startIndex, String str2, int str2StartIndex, int numChars)
- For both versions, startIndex specifies the index at which the region begins within the invoking String object.
- The String being compared is specified by str2.
- The index at which the comparison will start within str2 is specified by str2StartIndex.

- The length of the substring being compared is passed in numChars.
- In the second version, if ignoreCase is true, the case of the characters is ignored. Otherwise, case is significant

compareTo( ) Often, it is not enough to simply know whether two strings are identical. For sorting applications, you need to know which is less than, equal to, or greater than the next.

- The String method **compareTo( )** serves this purpose.
- It has this general form: **int compareTo(String str)** Here, str is the String being compared with the invoking String. The result of the comparison is returned and is interpreted, as shown here:
- Less than zero The invoking string is less than str.
- Greater than zero The invoking string is greater than str.
- Zero The two strings are equal.

```
// A bubble sort for Strings.
class SortString {
static String arr[] = { "Now", "is", "the", "time", "for", "all",
"good", "men", "to", "come", "to", "the", "aid", "of", "their",
"country" };
public static void main(String args[]) {
for(int j = 0; j < arr.length; j++) {
 for(int i = j + 1; i < arr.length; i++) {
if(arr[i].compareTo(arr[j]) < 0) {
 String t = arr[j]
arr[j] = arr[i];
arr[i] = t;
 }
 }
System.out.println(arr[j]);
}        }      } The output of this program is the list of words:
Now aid all come country for good is men of the the their time
to to
```

8. **Explain the difference between the indexOf() and lastIndexOf() methods in the String class. Give an example for each method to illustrate their functionality.**

The String class provides two methods that allow you to search a string for a specified character or substring:

- **indexOf( )** Searches for the first occurrence of a character or substring.
- **lastIndexOf( )** Searches for the last occurrence of a character or substring.
- To search for the first occurrence of a character, use **int indexOf(int ch)**
- To search for the last occurrence of a character, use **int lastIndexOf(int ch)** Here, ch is the character being sought.
- To search for the first or last occurrence of a substring, use **int indexOf(String str)**
  int lastIndexOf(String str) Here, str specifies the substring.
- You can specify a starting point for the search using these forms: **int indexOf(int ch, int startIndex)**
  int lastIndexOf(int ch, int startIndex)
  int indexOf(String str, int startIndex)
  int lastIndexOf(String str, int startIndex) Here, startIndex specifies the index at which point the search begins.
- For indexOf( ), the search runs from startIndex to the end of the string.
- For lastIndexOf( ), the search runs from startIndex to zero.
- The following example shows how to use the various index methods to search inside of Strings:

```
// Demonstrate indexOf() and lastIndexOf().
class indexOfDemo {
public static void main(String args[]) {
String s = "Now is the time for all good men " + "to come to the aid of their country.";
System.out.println(s);
System.out.println("indexOf(t) = " + s.indexOf('t'));
System.out.println("lastIndexOf(t) = " + s.lastIndexOf('t'));
System.out.println("indexOf(the) = " + s.indexOf("the"));
System.out.println("lastIndexOf(the)="+s.lastIndexOf("the"));
System.out.println("indexOf(t, 10) = " + s.indexOf('t', 10));
System.out.println("lastIndexOf(t, 60) = "+ s.lastIndexOf('t', 60));
System.out.println("indexOf(the, 10) = " + s.indexOf("the", 10));
```

System.out.println("lastIndexOf(the,60)= " + s.lastIndexOf("the", 60)); } }

Here is the output of this program:

Now is the time for all good men to come to the aid of their country.
indexOf(t) = 7
lastIndexOf(t) = 65
indexOf(the) = 7
lastIndexOf(the) = 55
indexOf(t, 10) = 11
lastIndexOf(t, 60) = 55
indexOf(the, 10) = 44
lastIndexOf(the, 60) = 55

9. **Describe two common approaches for modifying Strings in Java with an example**

substring( ) You can extract a substring using substring( ). It has two forms.

- The first is String **substring(int startIndex)** Here, startIndex specifies the index at which the substring will begin. This form returns a copy of the substring that begins at startIndex and runs to the end of the invoking string.
- The second form of substring( ) allows you to specify both the beginning and ending index of the substring: **String substring(int startIndex, int endIndex)** Here, startIndex specifies the beginning index, and endIndex specifies the stopping point.
- The string returned contains all the characters from the beginning index, up to, but not including, the ending index.
- // Substring replacement.

```
class StringReplace {
public static void main(String args[]) {
String org = "This is a test. This is, too.";
String search = "is";
String sub = "was";
String result = "";
int i;
do { // replace all matching substrings
System.out.println(org);
```

```
i = org.indexOf(search);
 if(i != -1) {
result = org.substring(0, i);
result = result + sub;
result = result + org.substring(i + search.length());
org = result; } }
while(i != -1); } }
```

The output from this program is shown here:

```
This is a test. This is, too.
Thwas is a test. This is, too.
Thwas was a test. This is, too.
Thwas was a test. Thwas is, too.
Thwas was a test. Thwas was, too.
```

10. Explain the use of charAt() and setCharAt() methods with suitable example.

charAt( ) and setCharAt( )

- The value of a single character can be obtained from a StringBuffer via the **charAt( )** method.
- You can set the value of a character within a StringBuffer using **setCharAt( ).**
- Their general forms are shown here:
  char charAt(int where)
  void setCharAt(int where, char ch)
- For charAt( ), where specifies the index of the character being obtained.
- For setCharAt( ), where specifies the index of the character being set, and ch specifies the new value of that character.
- For both methods, where must be nonnegative and must not specify a location beyond the end of the buffer.
- The following example demonstrates charAt( ) and setCharAt( ):

```
// Demonstrate charAt() and setCharAt().
class setCharAtDemo {
public static void main(String args[]) {
StringBuffer sb = new StringBuffer("Hello");
System.out.println("buffer before = " + sb);
System.out.println("charAt(1) before = " + sb.charAt(1));
sb.setCharAt(1, 'i');
sb.setLength(2);
```

```
System.out.println("buffer after = " + sb);
System.out.println("charAt(1) after = " + sb.charAt(1)); } }
```

Here is the output generated by this program:

```
buffer before = Hello
charAt(1) before = e
buffer after = Hi
charAt(1) after = i
```

11. Explain the purpose of the valueOf() method in the String class with an example

- The **valueOf( )** method converts data from its internal format into a human-readable form.
- It is a static method that is overloaded within String for all of Java's built-in types so that each type can be converted properly into a string.
- valueOf( ) is also overloaded for type Object, so an object of any class type you create can also be used as an argument.
- Here are a few of its forms:
  static String valueOf(double num)
  static String valueOf(long num)
  static String valueOf(Object ob)
  static String valueOf(char chars[ ])
- Any object that you pass to valueOf( ) will return the result of a call to the object's toString( ) method
- There is a special version of valueOf( ) that allows you to specify a subset of a char array.
- It has this general form: **static String valueOf(char chars[ ], int startIndex, int numChars)** Here, chars is the array that holds the characters, startIndex is the index into the array of characters at which the desired substring begins, and numChars specifies the length of the substring.

12. What is the use of replace() method? Explain.

replace( ): You can replace one set of characters with another set inside a StringBuffer object by calling replace( ).

- Its signature is shown here: **StringBuffer replace(int startIndex, int endIndex, String str)**
- The substring being replaced is specified by the indexes startIndex and endIndex.

- Thus, the substring at startIndex through endIndex1 is replaced.
- The replacement string is passed in – str.
- The resulting StringBuffer object is returned.
- The following program demonstrates replace( ):

```
// Demonstrate replace()
class replaceDemo {
 public static void main(String args[]) {
StringBuffer sb = new StringBuffer("This is a test.");
sb.replace(5, 7, "was");
System.out.println("After replace: " + sb); } }
```

Here is the output: After replace: This was a test.

13.With an example explain the four methods of StringBuffer class

Refer 10th, 12th answer

insert( ): The insert( ) method inserts one string into another.

- It is overloaded to accept values of all the simple types, plus Strings, Objects, and CharSequences.
- Like append( ), it calls String.valueOf( ) to obtain the string representation of the value it is called with.
- This string is then inserted into the invoking StringBuffer object.
- These are a few of its forms:

  StringBuffer insert(int index, String str)
  StringBuffer insert(int index, char ch)
  StringBuffer insert(int index, Object obj)

  Here, index specifies the index at which point the string will be inserted into the invoking StringBuffer object.

The following sample program inserts "like" between "I" and "Java":

```
// Demonstrate insert().
class insertDemo {
public static void main(String args[]) {
StringBuffer sb = new StringBuffer("I Java!");
sb.insert(2, "like ");
System.out.println(sb);
 }
} The output of this example is shown here: I like Java!
```

reverse( ) You can reverse the characters within a StringBuffer object using reverse( ), shown here: **StringBuffer reverse( )**

- This method returns the reversed object on which it was called.
- The following program demonstrates reverse( ):

```
// Using reverse() to reverse a StringBuffer.
class ReverseDemo {
public static void main(String args[]) {
StringBuffer s = new StringBuffer("abcdef");
System.out.println(s);
s.reverse();
System.out.println(s);
}
}
```

Here is the output produced by the program:

```
abcdef
fedcba
```

14. What is the usage of delete() and deleteCharAt() methods? Explain them using their syntax and an example

delete( ) and deleteCharAt( )

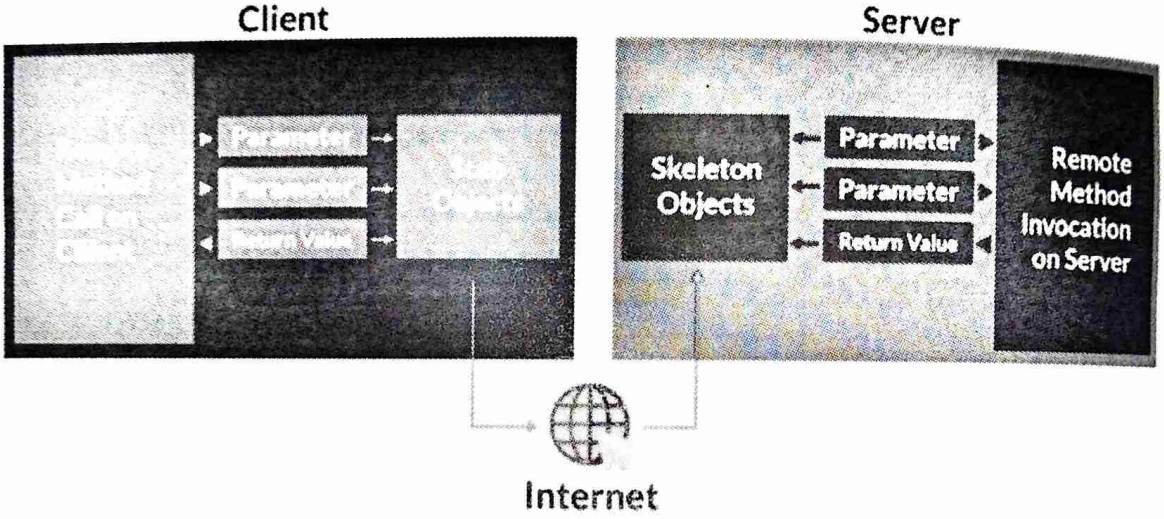
- You can delete characters within a StringBuffer by using the methods **delete( ) and deleteCharAt( ).**
- These methods are shown here:
  StringBuffer delete(int startIndex, int endIndex)
  StringBuffer deleteCharAt(int loc)
- The delete( ) method deletes a sequence of characters from the invoking object. Here, startIndex specifies the index of the first character to remove, and endIndex specifies an index one past the last character to remove.
- Thus, the substring deleted runs from startIndex to endIndex–1. The resulting StringBuffer object is returned.
- The deleteCharAt( ) method deletes the character at the index specified by loc. It returns the resulting StringBuffer object. Here is a program that demonstrates the delete( ) and deleteCharAt( ) methods:

```
// Demonstrate delete() and deleteCharAt()
class deleteDemo {
public static void main(String args[]) {
StringBuffer sb = new StringBuffer("This is a test.");
sb.delete(4, 7);
```

```
System.out.println("After delete: " + sb);
sb.deleteCharAt(0);
System.out.println("After deleteCharAt: " + sb);
}} The following output is produced:
After delete: This a test.
After deleteCharAt: his a test
```

15. Explain in detail the method that is used by the RMI client to connect to remote RMI servers?

The communication between client and server is handled by using two intermediate objects: Stub object (on client side) and Skeleton object (on server-side) as also can be depicted from below media as follows:



A high-level overview of how client and server communicate through remote objects using RMI is described below.

1. Define the Remote Interface:

- You define a Java interface that extends the java.rmi.Remote interface.
- Each method in this interface must declare java.rmi.RemoteException in its throws clause to handle remote method invocation errors.
- This interface defines the methods that the client can invoke on the remote object.

2. Implement the Remote Object:

- You implement the remote interface on the server side. This class will extend **java.rmi.server.UnicastRemoteObject** **or** **java.rmi.server.RemoteObject** and implement the methods defined in the remote interface.

- The server class provides the implementation for the methods declared in the remote interface.

3. **Create and Start the RMI Registry:**
   - The server creates an RMI registry, which acts as a central registry for remote objects. The registry listens for incoming requests on a specific port.
   - You can start the RMI registry from the command line using the rmiregistry tool provided with the JDK.
4. **Bind the Remote Object to the Registry:**
   - The server binds the remote object to the RMI registry using a unique name.
   - This makes the remote object accessible to clients by its name.
5. **Lookup the Remote Object on the Client Side:**
   - The client looks up the remote object in the RMI registry using the naming service **(java.rmi.Naming or java.rmi.registry.LocateRegistry).**
   - The client obtains a reference to the remote object, which it can then use to invoke remote methods
6. **Invoke Remote Methods:**
   - The client invokes methods on the remote object reference obtained from the RMI registry.
   - RMI handles the communication details, including parameter organizing, network communication, and error handling, transparently to the client.
7. **Handle Exceptions:**
   - Both the client and server should handle **java.rmi.RemoteException** and any other application-specific exceptions that may occur during remote method invocation

16. **Explain the concepts of object persistence and serialization in Java. Provide a brief overview of each concept, highlighting their significance in software development.**

Object persistence and serialization

Object persistence and serialization are two related concepts in Java that deal with the storage and retrieval of Java objects in a persistent form, typically to a file or over a network.

Object Persistence:

• Object persistence refers to the ability to store and retrieve objects beyond the lifetime of the application's execution.
• With object persistence, the state of an object can be saved to a persistent storage medium (such as a database, file system, or cloud storage) and later can be restored, allowing the application to work with the same data across different runs or even different instances of the application.
• Persistence is essential for applications that need to maintain data integrity, share data between multiple users or instances, or store data for long-term use.

Serialization:

• Serialization is the process of converting an object into a stream of bytes, which can be easily stored or transmitted and later reconstructed to create an identical copy of the original object.
• In Java, serialization is achieved by implementing the Serializable interface, which is a marker interface indicating that the class is serializable.
• Serializable objects can be written to an output stream (e.g., a file or network socket) using an **ObjectOutputStream**, and later read from an input stream using an ObjectInputStream.
• Serialization allows objects to be easily stored to disk, transmitted over a network, or saved to a database, enabling object persistence.

❖ Hence object persistence and serialization provide a powerful mechanism for storing and retrieving Java objects in a persistent form.
❖ They are commonly used in various applications, including data storage, caching, messaging systems, and distributed computing.
❖ However, it's essential to consider factors like performance, data integrity, and security when designing and implementing object persistence and serialization solutions

17.Explain four key challenges associated with distributed computing systems.

Challenges of Distributed Systems

While distributed systems offer many advantages, they also present some challenges that must be addressed. These challenges include:

- **Network latency:** The communication network in a distributed system can introduce latency, which can affect the performance of the system.
- **Distributed coordination:** Distributed systems require coordination among the nodes, which can be challenging due to the distributed nature of the system.
- **Security:** Distributed systems are more vulnerable to security threats than centralized systems due to the distributed nature of the system.
- **Data consistency:** Maintaining data consistency across multiple nodes in a distributed system can be challenging.

18. **Describe the three key components that make up a Distributed Computing System.**

Components There are several key components of a Distributed Computing System

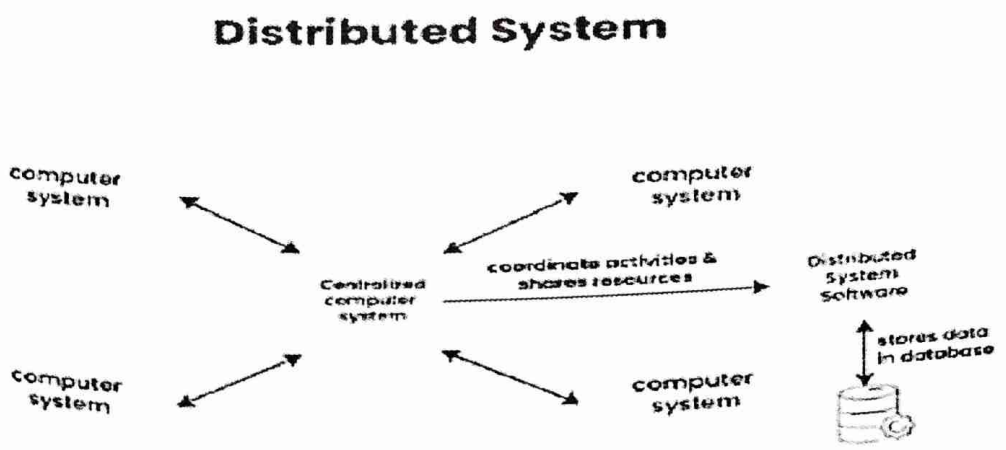
Devices or Systems: The devices or systems in a distributed system have their own processing capabilities and may also store and manage their own data.

Network: The network connects the devices or systems in the distributed system, allowing them to communicate and exchange data.

Resource Management: Distributed systems often have some type of resource management system in place to allocate and manage shared resources such as computing power, storage, and networking

19. **Explain how a Social Media platform can be considered a Distributed Computing System. Identify the key components involved and their roles.**

Example of a Distributed System

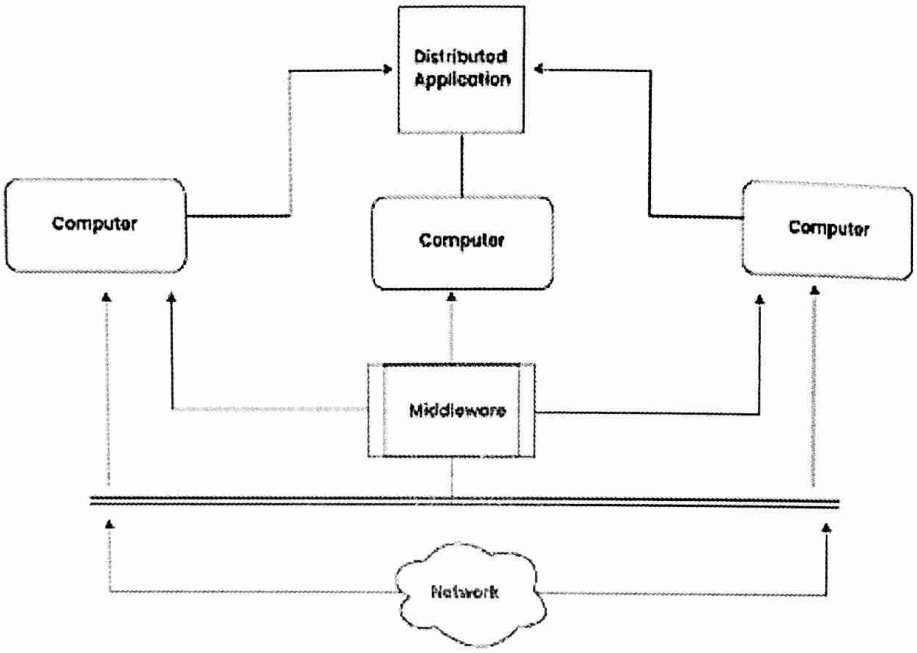


• Any Social Media can have its Centralized Computer Network as its Headquarters and computer systems that can be accessed by any user and using their services will be the Autonomous Systems in the Distributed System Architecture

Distributed System Software: This Software enables computers to coordinate their activities and to share the resources such as Hardware, Software, Data, etc.

• **Database:** It is used to store the processed data that are processed by each Node/System of the Distributed systems that are connected to the Centralized network

Working of Distributed System



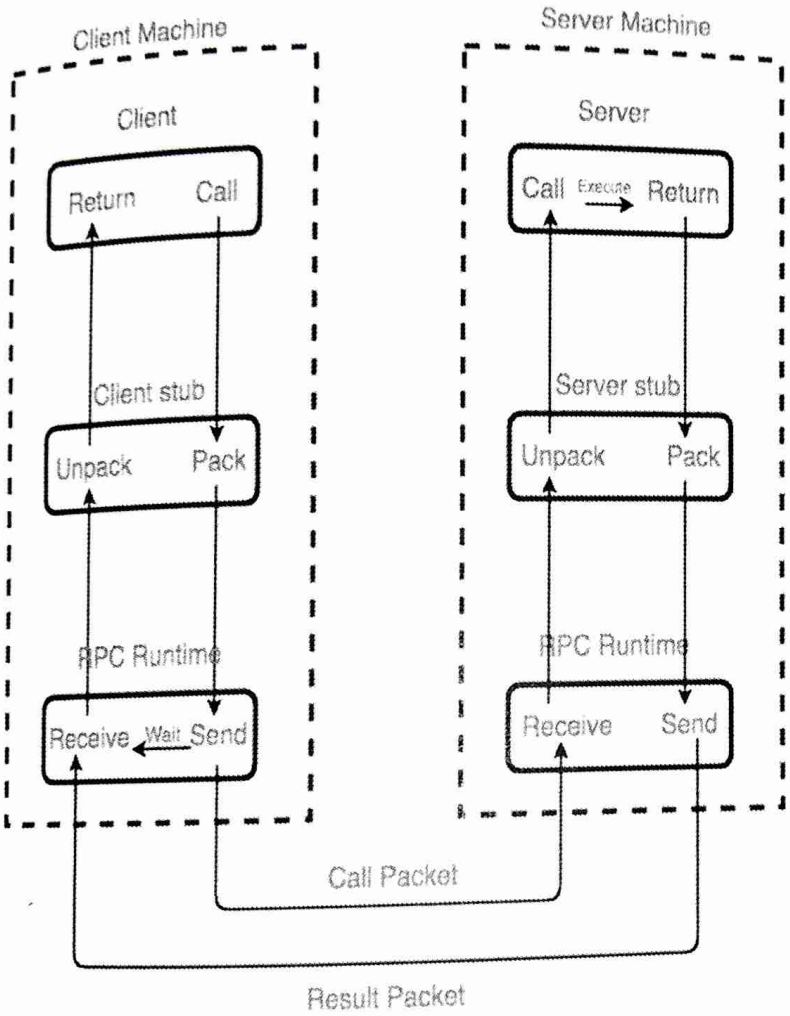
•As we can see that each Autonomous System has a common Application that can have its own data that is shared by the Centralized Database System.

• To Transfer the Data to Autonomous Systems, Centralized System should be having a Middleware Service and should be connected to a Network.

• Middleware Services enable some services which are not present in the local systems or centralized system default by acting as an interface between the Centralized System and the local systems. By using components of Middleware Services systems communicate and manage data.

• The Data which is been transferred through the database will be divided into segments or modules and shared with Autonomous systems for processing.
• The Data will be processed and then will be transferred to the Centralized system through the network and will be stored in the database

20. Explain the concept of Remote Procedure Calls (RPC) and its working mechanism with its five key elements



Implementation of RPC mechanism

Working of RPC: There are 5 elements used in the working of RPC:

- Client
- Client Stub
- RPC Runtime
- Server Stub
- Server

Client: The client process initiates RPC. The client makes a standard call, which triggers a correlated procedure in the client stub.

Client Stub: Stubs are used by RPC to achieve semantic transparency.

The client calls the client stub. Client stub does the following tasks:

- The first task performed by client stub is when it receives a request from a client, it packs(marshalls) the parameters and required specifications of remote/target procedure in a message.

- The second task performed by the client stub is upon receiving the result values after execution, it unpacks (unmarshalled) those results and sends them to the Client

RPC Runtime:
The RPC runtime is in charge of message transmission between client and server via the network.

- Retransmission, acknowledgement, routing, and encryption are all tasks performed by it.
- On the client-side, it receives the result values in a message from the server-side, and then it further sends it to the client stub whereas, on the server-side, RPC Runtime got the same message from the server stub when then it forwards to the client machine.
- It also accepts and forwards client machine call request messages to the server stub.

Server Stub: Server stub does the following tasks:

- The first task performed by server stub is that it unpacks(unmarshalled) the call request message which is received from the local RPC Runtime and makes a regular call to invoke the required procedure in the server.
- The second task performed by server stub is that when it receives the server's procedure execution result, it packs it into a message and asks the local RPC Runtime to transmit it to the client stub where it is unpacked.

Server: After receiving a call request from the client machine, the server stub passes it to the server. The execution of the required procedure is made by the server and finally, it returns the result to the server stub so that it can be passed to the client machine using the local RPC Runtime.

21. What is RMI? Explain its key components

The Remote Method Invocation (RMI) architecture in Java facilitates communication between Java objects residing in different Java Virtual Machines (JVMs) over a network. It allows objects to invoke methods on remote objects as if they were local objects. The RMI architecture typically involves several components and follows a client-server model:

1. Remote Interface: • The remote interface is a Java interface that defines the methods that can be invoked remotely by clients. It extends the java.rmi.Remote interface.
• Each method in the remote interface must declare java.rmi.RemoteException in its throws clause to handle remote method invocation errors.

2. Remote Object Implementation:

• The remote object implementation is a Java class that provides the actual implementation of the methods defined in the remote interface.

• This class typically extends **java.rmi.server.UnicastRemoteObject** or implements **java.rmi.Remote.**

• It is responsible for executing the methods invoked remotely by clients.

3. RMI Registry:

• The RMI registry is a simple naming service provided by Java RMI that allows clients to look up remote objects by name.

• It acts as a central registry where remote objects are bound to names, making them accessible to clients.

• The server creates an RMI registry, and clients use it to locate remote objects.

4. Client:

• The client is the application or component that initiates communication with remote objects.

It obtains a reference to the remote object from the RMI registry using the naming service provided by java.rmi.Naming or java.rmi.registry.LocateRegistry.

• Once it has the reference, the client can invoke methods on the remote object as if it were a local object.

5. Marshalling and Unmarshalling:

• RMI handles the marshalling (serialization) and unmarshalling (deserialization) of method parameters and return values transparently to the client and server.

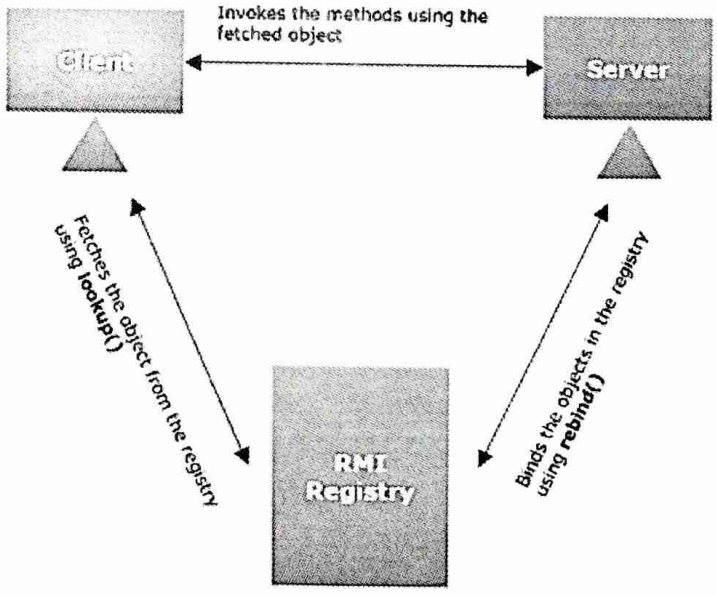
• Method parameters and return values are automatically serialized into a stream of bytes before being transmitted over the network and deserialized back into objects at the receiving end.

6. Network Communication: • RMI uses TCP/IP as the underlying network protocol for communication between the client and server JVMs.

• It establishes a connection between the client and server JVMs, allowing them to exchange method invocations and data

22.Write short note on RMI Registry

- RMI registry is a namespace on which all server objects are placed.
- Each time the server creates an object, it registers this object with the RMIregistry (using bind() or reBind() methods).
- These are registered using a unique name known as bind name.
- It allows remote clients to get a reference to these objects.
- To invoke a remote object, the client needs a reference of that object.
- At that time, the client fetches the object from the registry using its bind name (using lookup() method).
- The following illustration explains the entire process –



23.Outline the key steps involved in developing a basic Remote Method Invocation (RMI) application.

Step One: Enter and Compile the Source Code This application uses four source files.

- The first file, **AddServerIntf.java,** defines the remote interface that is provided by the server.
- It contains one method that accepts two double arguments and returns their sum.
- All remote interfaces must extend the Remote interface, which is part of java.rmi.
- Remote defines no members. Its purpose is simply to indicate that an interface uses remote methods.
- All remote methods can throw a RemoteException.

```
import java.rmi.*;
public interface AddserverIntf
{
public double add( double d1, double d2) throws
RemoteException;
}
```

- The second source file, **AddServerImpl.java**, implements the remote interface.
- The implementation of the add() method is direct.
- All remote objects must extend **UnicastRemoteObject,** which provides functionality that is needed to make objects available from remote machines

```
import java.rmi.*;
import java.rmi.server.*;
public class AddServerImpl extends UnicastRemoteObject
implements AddServerIntf {
public AddServerImpl() throws RemoteException() { }
public double add( double d1, double d2) throws
RemoteException { return d1+d2; }
}
```

- The third source file, **AddServer.java,** contains the main program for the server machine.
- Its primary function is to ***update the RMI registry*** on that machine.
- This is done by using the ***rebind() method of the Naming class (found in java.rmi).***
- That method associates a name with an object reference.
- The first argument to the rebind() method is a string that names the server as "**AddServer**".
- Its second argument is a reference to an instance of **AddServerImpl.**

```
import java.rmi.*;
```

```
import java.net.*;
public class AddServer {
public static void main(String args[]){
try{  AddServerImpl addServerImpl=new AddServerImpl();
      Naming.rebing("AddServer", addServerImpl);
      System.out.println("Server is running....");
}
catch(Exception e){  System.out.println("Excepiton"+e); }
}
}
```

- The fourth source file, **AddClient.java**, implements the client side of this distributed application.
- AddClient.java requires three command-line arguments.
- The first is the ***IP address or name of the server machine***.
- The second and third arguments are the two numbers that are to be summed.
- The application begins by forming a string that follows the URL syntax.
- This URL uses the **rmi protocol**.
- The string includes the IP address or name of the server and the string "AddServer".
- The program then invokes the lookup() method of the Naming class.
- This method accepts one argument, the rmi URL, and returns a reference to an object of type AddServerIntf.
- All remote method invocations can then be directed to this object.
- The program continues by displaying its arguments and then invokes the remote add() method.
- The sum is returned from this method and is then printed

```
import java.rmi.*;
public class AddClient{
public static void main(String args[]){
try{  String addServerURL= "rmi://"+args[0]+ "/AddServer";
AddServerIntf
addServerIntf=(AddServerIntf)Naming.lookup(addServerURL);
System.out.println("The First number is :+ args[1]);
Double d1=Double.valueOf(args[1].doubleValue();
System.out.println("The second number is :+ args[2]);
Double d2=Double.valueOf(args[2.doubleValue();
```

```
System.out.println("The sum is:"+addServerIntf.add(d1,d2));
}
catch(Exception e) {  System.out.println("Excepiton:"+ e);}
}
}
```

Compile all the four source code file using javac.

Step Two:

- Generate a Stub Next we need to generate the necessary stub.
- In the context of RMI, a stub is a Java object that resides on the client machine.
- Its function is to present the same interfaces as the remote server.
- If a response must be returned to the client, the process works in reverse.
- To generate a stub, we use a tool called the RMI compiler, which is invoked from the command line, as shown here: **rmic AddServerImpl**
- This command generates the file AddServerImpl_Stub.class.

Step Three:

- Install Files on the Client and Server Machines Copy **AddClient.class, AddServerImpl_Stub.class**, and **AddServerIntf.class** to a directory **Stub.class**, and **AddServer.class** to a directory on the server machine.

Step Four:

- Start the RMI Registry on the Server Machine Java SE 6 provides a program called **rmiregistry,** which executes on the server machine.
- It maps names to object references.
- start the RMI Registry from the command line, as shown here: **start rmiregistry**
- When this command returns, a new window has been created.
- This window must be open until experimenting with the RMI is done

Step Five:

- Start the Server The server code is started from the command line, as shown here: **java AddServer**

Step Six:

- **Start the Client** The Add Client software requires three arguments: the name or IP address of the server machine and the two numbers that are to be summed together.

- We may invoke it from the command line by using one of the two formats shown here: **java AddClient server1 8 9**

java AddClient 11.12.13.14 8 9

In the first line, the name of the server is provided.
The second line uses its IP address (11.12.13.14)

- We can try this example without actually having a remote server.
- To do so, simply install all of the programs on the same machine, start rmiregistry, start AddServer, and then execute AddClient using this command line:

java AddClient 127.0.0.1 8 9

Here, the address 127.0.0.1 is the "loop back" address for the local machine.

Sample output from this program is shown here:

The first number is: 8 The second number is: 9 The sum is: 17.

24. Explain the Advantages and Disadvantages distributed computing.

Advantages of the Distributed Computing System are:

- **Scalability:** Distributed systems are generally more scalable than centralized systems, as they can easily add new devices or systems to the network to increase processing and storage capacity

Reliability: Distributed systems are often more reliable than centralized systems, as they can continue to operate even if one device or system fails.

- **Flexibility:** Distributed systems are generally more flexible than centralized systems, as they can be configured and reconfigured more easily to meet changing computing needs.

There are a few limitations to Distributed Computing System

- **Complexity:** Distributed systems can be more complex than centralized systems, as they involve multiple devices or systems that need to be coordinated and managed.
- **Security:** It can be more challenging to secure a distributed system, as security measures must be implemented on each device or system to ensure the security of the entire system.
- **Performance:** Distributed systems may not offer the same level of performance as centralized systems, as processing and data storage is distributed across multiple devices or systems.